# COMP5339 Quiz 2 考场速查

W7-W10 · 题干触发词 + 判断规则 + SQL/公式 + 陷阱

## W7 时间三概念

- Instant=时刻；Interval=时长；Period=[start,end) 有边界时间段。
- 半开区间：start <= t AND t < end；连续窗口不重复边界。
- Valid time=现实何时为真；Transaction time=数据库何时知道；二者都有=Bitemporal。
- Current 查当前；Sequenced 按时间语义；Nonsequenced 把时间列当普通列。

□ timestamp 列本身不等于 temporal DBMS。

## W7 Temporal SQL / Range

- 期间重叠：a.start < b.end AND b.start < a.end。
- PostgreSQL tsrange/tstzrange：&& overlap；@> contains；GiST 加速 range 查询。
- 当前版本常用 end='infinity'；更新历史=关闭旧 period + 插入新版本。

```
WHERE ts >= :s AND ts < :e
valid_period && tstzrange(:s,:e,'[)')
```

## W7 SQL 做题顺序

- 逻辑顺序：FROM/JOIN -> WHERE筛行 -> GROUP BY分组 -> aggregate -> HAVING筛组 -> SELECT -> ORDER BY。
- 非聚合 SELECT 列必须在 GROUP BY；WHERE 不能筛 AVG/COUNT，HAVING 可以。
- CTE(WITH)给中间结果命名，便于检查；不保证缓存或更快。

```
WITH c AS (...)
SELECT k,AVG(v) FROM c
WHERE ts>=:s AND ts<:e
GROUP BY k HAVING COUNT(*)>=2;
```

## 题干词 -> 秒选

- real world -> VALID；recorded/knew -> TRANSACTION。
- aggregate condition -> HAVING；row condition -> WHERE。
- rare term -> TF-IDF；contextual -> BERT；generation -> GPT。
- topic/broker -> MQTT；micro-batch -> Spark；pipeline -> Flink。
- same-key order/replay -> Kafka；lightweight IoT -> MQTT。
- partition+latest -> CP；partition+must respond -> AP。

□ 多选逐项判断；ONLY/ALWAYS/AUTOMATICALLY 常为陷阱。

## W7 时序存储与数组

- Point-based=每时刻一行：易筛选/join/update。Sequence-based=一行含有序序列：易整段读取。
- array_agg(value ORDER BY ts) 保证顺序；unnest 展开。PostgreSQL 数组常从1开始。
- Spark collect_list 不保证顺序：先收集 (ts,value) struct，再 sort_array；explode 展开。
- 多个平行数组会错位；把整条记录作为 struct 一起排序。

```
array_agg(v ORDER BY ts)
sort_array(collect_list(named_struct('ts',ts,'v',v)))
```

□ Spark [] 从0开始；element_at(arr,1) 从1开始。

## W7 输入与检查

- 10位 Unix 常为秒，13位常为毫秒；先看数量级与日期范围。
- 显式列名 INSERT 更安全；append 增加，overwrite/UPDATE 可能破坏历史。
- 端到端：读入 -> schema/NULL/重复 -> 时间转换 -> 排序 -> 半开过滤 -> 聚合 -> 核对 count/min/max -> 画图。

## W8 数据与 ML 决策

- Structured=固定表；Semi-structured=JSON/XML 键存在但结构可变；Unstructured=文本/图像/音频。
- 有 label=Supervised：离散=classification，连续=regression；无 label=clustering/topic modelling。
- 正确顺序：split -> 只在 train fit 预处理/模型 -> validation 调参 -> test 一次最终评估。

□ 先用全数据标准化、选特征、建词表 = data leakage。

## W8 文本特征

- Tokenise分词；normalise统一形式；stemming粗截；lemmatisation按语言规则还原。清洗必须看任务，not 可能不能删。
- BoW/Count=词频，忽略词序；n-gram保留局部顺序但维度更高。
- TF-IDF=tf(t,d)*log(N/df(t))：降低到处出现的词，提高区分词；矩阵通常 sparse。

```
cos(A,B)=A·B/(||A|| ||B||)
Jaccard=|A∩B|/|A ∪ B|
```

## W8 向量与模型

- 文本稀疏向量常用 cosine：越大越相似；集合用 Jaccard；同尺度连续值才常用 Euclidean。
- Word2Vec=静态词向量；BERT=双向上下文表示，适合理解/分类；GPT=自回归 next-token，擅长生成。
- Embedding 把对象映射成稠密向量，向量近邻用于语义检索。

## W8 图像 / Metadata / Feature

- 图像常为 HxWxC；RGB=3 channel，grayscale=1；检查 RGB/BGR、0-255/0-1、尺寸与 dtype。
- CNN 学局部特征；ViT 将图像切 patch 用 attention；语义检索比较 embedding，不是逐像素。
- EXIF 可含时间/GPS/设备；文件修改时间不等于拍摄时间。
- Feature：interaction、polynomial、lag/rolling、encoding；时间特征绝不能看未来。

## W8 ExifTool 直接识别

- -TAG=读取指定标签；"-GPS*"匹配GPS标签；*.jpg匹配输入文件。
- -csv/-json决定输出格式；>是shell重定向，不是ExifTool参数。
- CreateDate不一定等于DateTimeOriginal/真实拍摄时间；metadata可缺失、错误或被改。

```
exiftool -common -csv *.jpg > metadata.csv
exiftool "-GPS*" photo.jpg
```

# COMP5339 Quiz 2 考场速查

W7-W10 · 题干触发词 + 判断规则 + SQL/公式 + 陷阱

## W9 Stream / DSMS / Pub-Sub

- Stream=持续到达、理论无界；需增量处理、窗口、状态、乱序/重复处理。
- DBMS：data persistent/query transient；DSMS：query persistent/data transient。
- Publisher -> topic -> broker -> subscriber；broker 解耦双方，但不自动保证 exactly-once。
- Broker(Kafka/MQTT)负责传输/缓冲；engine(Spark/Flink)负责窗口、状态、计算。

## W9 MQTT

- 轻量 Pub/Sub。订阅 sensor/+/temp：+匹配一层；sensor/#：#匹配后续任意层且只能在末尾。
- 消息到达触发 callback；retained message 与历史日志不是同一概念。
- QoS0=at most once；QoS1=at least once(可重复)；QoS2=协议单跳 exactly once(成本最高)。

```
client.subscribe('sensor/+/temp')
def on_message(client, userdata, msg): ...
```

 QoS2 不等于端到端业务恰好一次；sink 仍需事务/幂等/去重。

## W9 Kafka

- Topic 分 partitions；只保证 partition 内顺序，无跨 partition 全局顺序。
- offset=partition 内位置；同 consumer group 中一个 partition 同时给一个 consumer。
- key 决定分区可保同 key 顺序；consumer 数>partition 数时有人空闲。
- Kafka 是持久化可重放日志，复制提高容错。

## W9 状态与窗口

- Stateless：map/filter 只看当前事件。Stateful：count/dedup/window/running total/join 需历史状态。
- Tumbling=固定不重叠；Sliding=固定可重叠；Session=活动间隔；Count=按条数。
- size=统计范围；slide=输出频率；state 需 checkpoint、TTL/清理。

## W9 三种时间 + Watermark

- Event time=源头发生；Ingestion time=进入平台；Processing time=算子处理。业务乱序分析优先 event time。
- Watermark≈max observed event time - tolerated out-of-orderness，用于关闭窗口与清状态。
- 新事件 event_time <= watermark 通常为 late；可丢弃、更新结果或 side output，取决于配置。
- 等更久：结果更完整但延迟/状态更大。

 watermark 是进度估计，不保证之前绝不会再来数据。

## W9 Join / 框架

- Stream-table：用维表 enrichment；表版本决定用最新值还是历史一致值。
- Stream-stream：按 key + 时间范围，否则两边状态可能无限增长。
- Spark Structured Streaming：micro-batch为主、批流统一；Flink：原生逐事件、event-time/state强。

```
ON a.key=b.key
AND b.ts BETWEEN a.ts-INTERVAL '5 min' AND a.ts+INTERVAL
'5 min'
```

## W9 Tutorial 一条主线

- CSV row(dict) -> json.dumps(string) -> publish(topic) -> broker -> subscriber。
- msg.payload(bytes) -> decode(string) -> json.loads(dict) -> float -> deque(maxlen=N)/plot。
- 先开receiver并订阅，再开publisher；QoS0+retain=False可能错过早期消息。
- Local CSV replay只验证callback/plot，不证明真实MQTT连接。

```
client.on_message=on_message
client.connect(broker,1883)
client.loop_forever()
```

## W10 扩展与存储

- Vertical=加强单机；Horizontal=加节点；Elasticity=自动伸缩。Speedup=同一工作更快；Scaleup=数据/资源同比增长而时间近似不变。
- Row store：整行写/点查；Column store：少数列扫描/聚合；Parquet=列式文件；Wide-column=数据库模型，不是同一概念。
- 数据本地性减少网络；shuffle、串行协调、倾斜常是瓶颈。

## W10 Partition / Replication / CAP

- Range partition利范围查但易热点；Hash较均匀但范围散开；好 key 均匀且匹配查询。
- Replication提高可用/读扩展：sync一致但慢；async快但故障可丢未复制写。复制不是备份。
- CAP：发生 network partition(P) 时在 Consistency(C) 与 Availability(A)间取舍；不是平时永久‘三选二’。

## W10 Mongo / HDFS / MapReduce

- Mongo：mongos路由；config servers存元数据；shards存数据且常为replica sets；无 shard key 易 scatter-gather。
- HDFS：NameNode管文件/block位置元数据；DataNode存实际 blocks；适合大文件顺序吞吐，不适合大量小文件。
- Map -> (k,v)；shuffle/sort 按 key 重分布；reduce 聚合。shuffle 常最贵。

## W10 Spark 必写

- Driver 建 DAG/调度；executors 处理 partitions；transformation 惰性，action 才执行。
- map/filter/select=transformation；count/collect/write=action；宽依赖常触发 shuffle/stage。
- 优先内建 DataFrame 函数；大数据不要 collect()/toPandas() 到 driver。

```
df.filter(F.col('x')>0)\
.groupBy('k').agg(F.avg('v'))
df.createOrReplaceTempView('t')
spark.sql('SELECT ... FROM t')
```